

Plan d'action CI/CD pour viser la meilleure note

Projet Ynov - déploiement automatique, quality gates, preuves et soutenance

Objectif: garder le code applicatif stable et concentrer le travail restant sur la configuration CI/CD, le déploiement externe, les preuves de fonctionnement et la documentation DevOps.

Decision recommandée: GitHub Actions + GitHub Container Registry + Render

Cette option évite Jenkins et un VPS payant, tout en respectant le besoin de CI/CD, de build Docker, de push vers un registry et de déploiement sur un environnement externe. Render doit être configuré comme environnement de démonstration, avec un deploy hook appelé seulement après la réussite des tests, du coverage, des scans et du push Docker.

1. Ce que les modalités attendent vraiment

- Une pipeline CI/CD complète autour de l'application, adaptée au besoin réel du projet.
- Des tests automatiques sont exécutés dans la pipeline, avec mesure de couverture de code.
- Une politique Git: branches, Pull Request, quality gate et conventions de commit.
- Une application conteneurisée avec Docker, avec image builder puis poussée vers un registry.
- Un déploiement final sur un environnement réel externe, donc hors poste local.
- Quatre stages majeurs visibles dans l'outil CI/CD, chacun découpé en sous-étapes logiques.
- Une documentation DevOps claire, avec schéma d'architecture, configuration, secrets, pipeline et justification des choix.
- Aucun secret exposé dans le dépôt.

Important: Jenkins est un choix possible, pas une obligation. GitHub Actions suffit si la pipeline est lisible, automatisée et démontrable.

2. Architecture cible sans VPS payant

Bloc	Choix conseille	Justification pour l'evaluation
Gestion de code	GitHub	Dépôt, Pull Requests, protection de branche, historique clair des commits.
CI/CD	GitHub Actions	Intégré à GitHub, déclenchement automatique sur PR et push, stages visibles dans l'interface.
Registry Docker	GHCR - GitHub Container Registry	Image Docker versionnée et rattachée au dépôt. Pas besoin de Docker Hub.
Hebergement externe	Render	Environnement externe gratuit ou de démonstration, compatible Docker et déploiement par image registry.
Base de donnees	Render Postgres ou equivalent gratuit	Base externe separee de la machine locale, variables en secrets.
Securite	Gitleaks + Trivy	Détection de secrets et vulnérabilités image avant déploiement.
Verification post-deployment	Smoke tests HTTP	Preuve objective que l'application déployée répond après la pipeline.

Flux cible : “Commit” ou “Pull Request” → GitHub Actions → tests + coverage → build Docker → scan → push GHCR → deploy hook Render → smoke tests → URL publique validee.

3. Pipeline automatique a presenter

Stage visible	Objectif	Sous-étapes attendues	Condition de passage
1. Quality gate	Bloquer les changements non conformes avant les tests lourds.	Checkout, verification format, lint, commitlint sur PR, detection de secrets.	Aucun secret détecte, conventions de commit respectées, lint OK.
2. Tests et coverage	Prouver que le projet reste fonctionnel.	Install backend/frontend, tests unitaires, generation coverage, publication artefacts.	Tests OK et seuil minimal de couverture atteint.
3. Build et security scan	Valider que l'application est déployable en conteneur.	Build Docker backend/frontend, tags versionnes, scan Trivy des images.	Images buildables et vulnérabilités critiques traitées ou justifiées.
4. Push et deploy	Publier puis déployer automatiquement hors local.	Login GHCR, push images, appel deploy hook Render, smoke tests HTTP.	Images disponibles dans GHCR et application joignable en ligne.

Le stage 4 doit s'exécuter uniquement sur push vers main, après merge d'une PR. Sur Pull Request, il faut lancer quality gate + tests + build + scan, mais éviter le déploiement de production.

4. Configuration GitHub a faire

- ☒ Créer le dépôt GitHub et pousser la version stable du projet.
- ☒ Activer GitHub Actions.
- ☐ Dans Settings -> Actions -> Général, autoriser les workflows a lire le dépôt et à écrire dans Packages si GHCR est utilisé.
- ☐ Protéger la branche main: Pull Request obligatoire, checks obligatoires, interdiction de push direct.
- ☐ Ajouter un template de Pull Request pour forcer la vérification: tests, coverage, Docker, documentation, secrets.
- ☐ Conserver une branche de démonstration prête, par exemple demo/pipeline-proof, avec une petite modification documentaire.
- ☐ Vérifier que le pipeline se lance automatiquement sur pull request et push main.

5. Secrets et variables à configurer

Secret GitHub	Utilisation	Remarque
RENDER_DEPLOY_HOOK_BACKEND	Declenche le re-déploiement backend Render apres push GHCR.	Valeur fournie par Render, à stocker uniquement en secret.
RENDER_DEPLOY_HOOK_FRONTEND	Declenche le redéploiement frontend Render apres push GHCR.	Utile si frontend déployé comme image Docker séparée.
PROD_FRONTEND_URL	Smoke test du frontend après déploiement.	Exemple: URL publique Render du frontend.
PROD_BACKEND_URL	Smoke test API après déploiement.	Exemple: URL publique Render du backend.
DATABASE_URL	Connexion backend vers la base externe.	Plutôt dans Render si l'application lit cette variable au runtime.

POSTGRES_USER / POSTGRES_PASSWORD / POSTGRES_DB	Configuration DB si le projet les utilise.	Jamais dans le dépôt. Utiliser .env.exemple pour documenter les noms.
CORS_ORIGIN	Autorise le frontend public a appeler le backend.	Mettre l'URL exacte du frontend Render.
GHCR_TOKEN	Pull image GHCR depuis Render si image privée.	Facultatif si les packages GHCR sont publics.

Règle de notation: un fichier .env.exemple est attendu pour documenter les variables, mais le vrai .env et les tokens restent exclus du dépôt.

6. Configuration Render conseillée

1. Créer un compte Render.
2. Créer une base Render Postgres ou utiliser une base gratuite équivalente.
3. Créer un service Web backend depuis une image Docker GHCR prébuild.
4. Créer un service Web frontend depuis une image Docker GHCR prebuild, ou un Static Site si le frontend est purement statique.
5. Désactiver l'auto-déploiement direct depuis Git quand c'est possible, afin que GitHub Actions reste le vrai chef d'orchestre du déploiement.
6. Ajouter les variables d'environnement de production dans Render: DATABASE_URL, CORS_ORIGIN, NODE_ENV, ports, etc.
7. Si l'image GHCR est privée, ajouter dans Render un credential registry avec un token GitHub read:packages.
8. Créer les deploy hooks Render et les copier dans les secrets GitHub.
9. Lancer un premier déploiement manuel Render uniquement pour valider la configuration initiale. La soutenance devra ensuite montrer un déclenchement automatique via commit ou PR.

7. Vérifications automatiques a ajouter dans la pipeline

Verification	Commande ou principe	Interet pour la note
Tests backend	npm test ou npm run test:coverage dans backend	Couvre l'exigence tests automatisés + coverage.
Tests frontend	CI=true npm test -- --coverage ou script equivalent	Montre que les deux parties du projet sont vérifiées.
Coverage threshold	Seuil minimum configuré dans package.json ou script CI	Transforme le coverage en quality gate.
Docker build	docker build backend et frontend	Prouve que les images sont constructibles.
Scan secrets	Git Leaks sur le dépôt	Prouve la prévention de fuite de secrets.
Scan image	Trivy sur les images Docker	Montre une approche sécurité supply chain.
Push registry	docker push ghcr.io/...:sha et :latest	Valide l'exigence build + push image Docker.
Deploy hook	curl POST vers deploy hook Render	Déclenche le déploiement externe uniquement après CI OK.
Smoke test frontend	curl -f \$PROD_FRONTEND_URL	Prouve que l'UI est accessible après deploy.
Smoke test API	curl -f \$PROD_BACKEND_URL/api/health ou endpoint existant	Prouve que le backend répond dans l'environnement externe.

Si /api/health existe deja, l'utiliser. Si aucun endpoint de healthcheck existe et que vous ne voulez pas toucher au code, utiliser un endpoint public existant, par exemple une route GET stable.

8. Ce qui peut être fait sans toucher au code applicatif

- Configurer GitHub Actions, les secrets, les permissions GHCR et les protections de branche.
- Créer les services Render, la base externe et les variables d'environnement.
- Documenter l'architecture, les choix techniques, les limites et le plan de rollback dans le README.
- Ajouter ou ajuster les fichiers DevOps: workflow YAML, .env.example, PR template, dependabot, commitlint, docs CI/CD.
- Préparer la démonstration live avec une PR documentaire.

Point de vigilance: si le frontend contient une URL backend hardcodée vers localhost, il faudra au minimum passer cette URL par variable d'environnement ou configuration de build. Ce n'est pas une évolution fonctionnelle, mais une correction de configuration de déploiement.

9. Preuves à collecter avant la soutenance

- ☐ Capture ou lien vers une Pull Request avec checks verts.
- ☐ Capture ou lien vers une exécution complète GitHub Actions sur main.
- ☐ Lien GHCR montrant les images Docker publiées avec tags sha et latest.
- ☐ Lien Render frontend public.
- ☐ Lien Render backend public ou endpoint /api/health.
- ☐ Preuve que les secrets sont dans GitHub/Render et absents du dépôt.
- ☐ README avec schéma d'architecture et explication des 4 stages.
- ☐ Rapport coverage backend/frontend ou artefacts GitHub Actions.
- ☐ Resultat Trivy/Gitleaks dans les logs de pipeline.
- ☐ Procédure de rollback courte: redéploier une image taggée précédente ou revert + pipeline.

10. Scenario de démonstration live

1. Avant le passage, ouvrir GitHub Actions, la Pull Request de demo, Render et les URLs publiques.
2. Préparer une PR contenant une modification documentaire sans risque, par exemple une phrase dans README.md.
3. Montrer que la PR déclenche automatiquement la pipeline, sans bouton manuel.
4. Commenter rapidement les stages: quality gate, tests + coverage, Docker + scan, push + deploy.
5. Merger la PR vers main si le temps et le format de soutenance le permettent.
6. Montrer le pipeline main qui pousse les images GHCR puis déclenche Render.
7. Terminer par les smoke tests et l'application accessible via URL publique.

Prevoir un plan B: une execution reussie recente dans l'historique GitHub Actions, ouverte dans un onglet. Le live doit être préparé car le brief indique que la démonstration ne doit pas planter.

11. Mapping direct avec le bareme

Critere	Points	Action a montrer
Pertinence des choix	4	README expliquant GitHub Actions vs Jenkins, Render vs VPS, GHCR vs Docker Hub, Docker Compose/Kubernetes ecarte ou justifie.
Pipeline fonctionnelle	5	Execution complete sans erreur, stages visibles, deploiement externe et smoke tests reussis.
Qualité d'implémentation	4	Workflow lisible, commentaires utiles, secrets absents, variables documentees, logs comprehensibles.
Tests et Code Quality	3	Tests backend/frontend, coverage, quality gates, lint, commitlint.
Documentation	4	README DevOps complet avec schema d architecture, setup local, setup CI/CD, secrets, deploiement, rollback.

12. Checklist chronologique

Phase A - Depot et qualite

- ☐ Pousser le projet sur GitHub.
- ☐ Verifier que .env, node_modules, coverage et fichiers sensibles sont ignores.
- ☐ Configurer commitlint et le template de PR.
- ☐ Activer la protection de main.
- ☐ Lancer une PR de test et verifier les checks.

Phase B - Registry Docker

- ☐ Configurer les permissions packages: write dans GitHub Actions.
- ☐ Pousser les images backend/frontend vers GHCR.
- ☐ Verifier que les images sont visibles dans Packages.
- ☐ Choisir public packages ou token read:packages pour Render.

Phase C - Render

- ☐ Creer la base de donnees externe.
- ☐ Creer le service backend depuis image GHCR.
- ☐ Creer le service frontend depuis image GHCR ou static site.
- ☐ Configurer les variables d environnement.
- ☐ Recuperer les deploy hooks et les stocker dans GitHub Secrets.
- ☐ Valider un premier deploiement.

Phase D - Deploiement automatique

- ☐ Faire un commit sur une branche de demo.
- ☐ Ouvrir une PR et montrer le declenchement automatique.
- ☐ Verifier que le deploy ne se fait que sur main.
- ☐ Merger vers main et suivre la pipeline complete.
- ☐ Verifier les URLs publiques avec les smoke tests.

Phase E - Documentation et oral

- [] Compléter README avec schema d architecture Mermaid.
- [] Documenter les secrets sans afficher leurs valeurs.
- [] Ajouter la procedure de deploiement et rollback.
- [] Preparer 5 phrases de justification technique.
- [] Repeter la demonstration chronometree.

13. Phrases de justification a connaitre

- Nous avons choisi GitHub Actions car il est integre au depot, declaratif en YAML et suffisant pour automatiser tests, build, scan, publication d images et deploiement.
- Nous avons ecarte Jenkins car il aurait impose un serveur CI a administrer, alors que le besoin du projet est couvert par GitHub Actions avec moins de charge operationnelle.
- Nous avons choisi GHCR car il est directement lie a GitHub et permet de tracer les images Docker produites par la pipeline.
- Nous avons choisi Render pour eviter un VPS payant tout en deployant sur un environnement externe reel.
- Nous declenchons Render par deploy hook apres les tests et le push Docker afin que le deploiement soit conditionne a une pipeline verte.
- Nous avons ajoute des smoke tests pour verifier que le deploiement n est pas seulement execute, mais aussi utilisable.
- Kubernetes et Terraform ont ete ecartes car le sujet les rend optionnels et l architecture du projet ne justifie pas cette complexite.

14. Definition de done avant rendu

- [] Une PR declenche automatiquement la CI.
- [] Un push ou merge sur main declenche la CI puis le CD.
- [] Les 4 stages sont visibles dans GitHub Actions.
- [] Les tests et le coverage backend/frontend sont visibles dans les logs.
- [] Les images Docker sont poussees dans GHCR.
- [] Render redeploie l application depuis l image publiee.
- [] Les smoke tests valident frontend et backend apres deploiement.
- [] Le README permet a un nouveau dev de comprendre toute la CI/CD sans explication orale.
- [] Aucun secret n apparait dans le depot, les logs, le README ou les fichiers de configuration.
- [] Une demonstration live a deja ete repete au moins une fois de bout en bout.

Sources utilisees

- Modalites Evaluation CI/CD fournies dans le PDF du sujet: exigences techniques, documentation, soutenance et bareme.
- Documentation GitHub Actions: declenchements push et pull_request, deploiement vers plateformes tierces, GHCR.
- Documentation Render: deploiement gratuit, deploy hooks, Docker image prebuild et registry GHCR.